Programming Project (NEA) — Student Template & Workbook

Name: ____ Candidate no.: __ Project title: _______

Type your project write-up into this template. The prompts and checklists tell you what the examiner is looking for at each stage. Delete the grey prompt text as you complete each part. Worth 70 marks / 20% of the A Level.

> The four stages are **Analysis → Design → Development → Evaluation**. Keep evidence (screenshots, test results) as you go — don't leave it to the end.

Stage 1 — Analysis

1.1 Problem identification

What is the problem? Who is it for? Why is it suited to a computational (programmed) solution?

1.2 Stakeholders

Name your stakeholder(s) and describe their needs. Include evidence of research — an interview, questionnaire or observation.

Stakeholder	Their need	How you found out

1.3 Research into existing solutions

Look at 1–2 similar solutions. What features work well? What will you do differently?

1.4 Requirements & measurable success criteria

Each criterion must be specific and testable — you'll measure against these in the Evaluation.

#	Success criterion (measurable)	Why it's needed (link to stakeholder)
1		
2		
3		

Checklist: ☐ real problem justified ☐ named stakeholder + research ☐ measurable, testable criteria ☐ computational focus shown (decomposition/abstraction).

Stage 2 — Design

2.1 Decomposition / structure

Break the problem into modules. A structure chart or module list.

2.2 Key algorithms

Flowcharts or pseudocode for the important/complex parts (this is where your computational complexity shows).

```
(pseudocode / algorithm here)
```

2.3 Data structures & data

Variables, records, arrays, files or database schema — and why each choice.

2.4 User interface

Wireframes / screen designs and how the user navigates.

2.5 Test plan

Plan tests now: typical, boundary and erroneous data, mapped to your success criteria.

Test	Data (type)	Purpose	Expected result	Success criterion

Checklist: ☐ decomposition ☐ justified algorithms ☐ justified data structures ☐ UI design ☐ test plan with boundary/erroneous data, linked to criteria.

Stage 3 — Development

Develop in clear iterations. For each iteration: what you built, annotated code screenshots, and testing of that part with evidence.

Iteration 1 — [name]

Built: … **Annotated code:** *(screenshot with notes on what each part does)* **Testing this iteration:** *(what you tested, evidence, any bugs found & fixed)*

Iteration 2 — [name]

…

(add iterations as needed)

Good-practice reminders: meaningful names · modular subroutines · validation & error handling (robustness) · comments · show the *complexity you designed* actually working · evidence of review → refinement between iterations.

Checklist: ☐ iterative development ☐ annotated evidence ☐ testing throughout (not just at the end) ☐ robust to bad input ☐ complexity implemented.

Stage 4 — Evaluation

4.1 Testing against success criteria

Test each criterion with evidence (screenshots / test table), including boundary and erroneous data.

Criterion	Met? (Y / Partly / N)	Evidence (test #)	Comment
1			

4.2 Evaluation of the solution

How well did it meet each requirement — including ones only partly met (honesty scores)? Use stakeholder feedback.

4.3 Usability

Evaluate usability for your stakeholder.

4.4 Improvements / future development

Specific, justified improvements.

Checklist: ☐ tested against every criterion with evidence ☐ honest evaluation (incl. partial) ☐ stakeholder feedback ☐ usability ☐ justified improvements — all mapped back to the Analysis.

Final self-check before submission

☐ Every success criterion is measurable and evaluated · ☐ Development shows *develop → test → refine* iterations · ☐ Code is annotated and robust · ☐ Evaluation maps back to Analysis · ☐ All your own work, sources acknowledged (follow your centre's authenticity rules).

See the full NEA Guide in this component for detailed mark-band advice.